

## ► DATABASE : MySQL

### What is Database?

collection of data in a format that can be easily accessed.

### Why databases?

- Can store large data
- features like security, scalability etc
- Easier to insert, update or delete data, search as well

## ► SQL VS NOSQL?

### SQL

Relational Database  
(data stored in Tables)

Eg: MySQL, Oracle, PostgreSQL etc

### NOSQL

Non-relational Database  
(data stored in document/key/value/graphs etc)

Eg: MongoDB, Cassandra, Neo4j etc

## ► What is SQL?

is a programming language used to interact with relational databases

USER

| Id | Name   | Email  | Follower | Follow |
|----|--------|--------|----------|--------|
| 1  | Adam   | xyz -- | 123      | 145    |
| 2  | Bob    | - - -  | 200      | 308    |
| 3  | Donald | - - -  | 300      | 105    |

- columns → design (schema)  
[id, name, email, followers, following]
- rows → tuple (info of one individual)

## ► Basic commands

CREATE DATABASE db-name;

DROP DATABASE db-name;

USE db-name;

## Our 1st table:

CREATE TABLE table-name(

column-name1 datatype  
constraint,

column-name2 datatype  
constraint,

column-name3 datatype  
constraint;

);

Eg: CREATE TABLE student(

roll-no INT,

name VARCHAR(30),

age INT

INSERT INTO student

VALUES

(101, "adam", 12),

(102, "bob", 14);

SELECT \* FROM student;

## Database Queries

CREATE DATABASE db-name;  
 CREATE DATABASE IF NOT EXISTS db-name;  
 DROP DATABASE db-name;  
 DROP DATABASE IF EXISTS db-name;  
 SHOW DATABASES;  
 SHOW TABLES;

## Table Queries

1. **Create** : defines schema/ columns

CREATE TABLE table\_name(  
 column\_name1 datatype constraint,  
 column\_name2 datatype constraint,  
 );

## Constraints

Rules for data in a table

NOT NULL columns cannot have a null value

UNIQUE all values in column are different

DEFAULT sets the default value of a column

CHECK it can limit the values allowed in columns

| Datatype | Description                                                     |             |
|----------|-----------------------------------------------------------------|-------------|
| CHAR     | fixed length                                                    | CHAR        |
| VARCHAR  | given length                                                    | VARCHAR     |
| BLOB     | binary large object                                             | BLOB (1000) |
| INT      | integer                                                         | INT         |
| TINYINT  | small integer                                                   | TINYINT     |
| BIGINT   | big integer                                                     | BIGINT      |
| BIT      | can store 2-bit values range 1 to 64                            | BIT(2)      |
| FLOAT    | decimal                                                         | FLOAT       |
| DOUBLE   | decimal -24 to 53 digit precision                               | DOUBLE      |
| BOOLEAN  | boolean values 0 or 1                                           | BOOLEAN     |
| DATE     | date format of YYYY-MM-DD ranging from 1000-01-01 to 9999-12-31 | DATE        |
| YEAR     | 4 digit (1901-2155)                                             | YEAR        |

NOTE: UNSIGNED



only positive integer

-128 to 127 ⇒ 0 to 255



1 to 200 ←

salary INT <sup>ε</sup> DEFAULT 25000  
CONSTRAINT age-check  
(age >= 18 AND city = "Delhi")

### • Constraints

• **PRIMARY KEY** makes a column unique & not null but used only for one

```
CREATE TABLE tem (  
  id INT not null,  
  PRIMARY KEY (id)  
);
```

• **FOREIGN KEY** prevents actions that would destroy links between tables

```
CREATE TABLES temp (  
  cust_id INT,  
  FOREIGN KEY (cust_id)  
  references customer(id)  
);
```

2. **Insert**: insert values in a table

```
INSERT INTO table_name  
(colname1, colname2) VALUES  
(col1-v1, col2-v1),  
(col1-v2, col2-v2);
```

### ► What are keys?

keys are special columns in the table

• **Primary key**: columns (or set of columns) in a table that uniquely identifies each row. (a unique id)

There is only 1 PK and it should be not null

• **Foreign key**: column (or set of columns) in the table that refers to the primary key in table  
FKs can have duplicate & null values

There can be multiple FKs

3. **Select**: select and show the elements in a table

```
SELECT col1, col2 FROM table-  
name;
```

```
SELECT * FROM table_name;  
SELECT DISTINCT remove duplicate field name  
FROM tablename;
```

### ► where clause

to define some conditions

```
SELECT * FROM table_name  
WHERE condition;
```

► Operators that can be used with "where" clauses

1. Arithmetic Operators:

+ , - , \* , / , %

SELECT \* FROM table\_name  
WHERE condition;

↓

SELECT name, following  
FROM user WHERE age > 18;

2. Comparison Operators:

= , != , > , >= , < , <=

3. Logical Operators:

AND , OR , NOT , IN , BETWEEN  
ALL , LIKE ANY

4. Bitwise Operators:

& (Bitwise AND),

| (Bitwise OR)

► Limit Clause

sets an upper limit on  
number of (tuples) rows to be  
returned

SELECT col1, col2 FROM  
table\_name  
LIMIT number;

► Order by clause

To sort in ASC or  
DESC order

SELECT col1, col2 FROM  
table\_name ORDER BY  
col\_name(s) ASC;

NOTE: by default ASC

► Aggregate Functions

Aggregate Functions perform  
a calculation on a set of  
values, and return a single  
value

• COUNT()

• MAX()

• MIN()

• SUM()

• AVG()

Example:

SELECT max(marks)  
FROM student;

► Group by Clause

Groups rows that have the  
same values into summary  
rows.

It collects data from  
multiple records and  
groups the results by  
one or more columns.

```
SELECT col1, col2  
FROM table_name  
GROUP BY colname(s);
```

\* Generally we use group by with some aggregation function

```
SELECT age, count(id)  
FROM user
```

```
GROUP BY age;
```

### ► Having clause

Similar to WHERE i.e. applies some condition on rows.

But is used when we want to apply any condition after grouping.

Example :

```
SELECT col1, col2  
FROM table_name  
GROUP BY colname(s)  
HAVING condition;
```

```
SELECT age, max(followers)  
FROM user
```

```
GROUP BY age
```

```
HAVING max(followers) >  
200;
```

• WHERE is for the table,  
HAVING is for a group

• Grouping is necessary for HAVING

### ► General Order

```
SELECT column(s)  
FROM table_name  
WHERE condition  
GROUP BY column(s)  
HAVING condition  
ORDER BY column(s) ASC;
```

4. Update : to update/  
manipulate the existing  
rows

```
UPDATE table_name  
SET col1 = val1, col2 = val2  
WHERE condition;
```

Eg : UPDATE user  
SET followers = 600  
WHERE age = 16;

NOTE : SET SQL\_SAFE\_UPDATES = 0; is required to run UPDATE commands

5. Delete : to delete  
existing rows

```
DELETE FROM table_name  
WHERE condition;
```

5. Alter : to change the schema

**ADD COLUMN**

ALTER TABLE table-name

ADD COLUMN column-name

datatype constraints;

**DROP COLUMN**

ALTER TABLE table-name

DROP COLUMN column-name;

**RENAME Column**

ALTER TABLE table-name

RENAME TO new table name;

**CHANGE column (rename)**

ALTER TABLE table-name

CHANGE COLUMN old-name

new-name new-datatype

new-constraints;

**MODIFY column (modify**

**datatype / constraint)**

ALTER TABLE table-name

MODIFY col-name new-datatype

new-constraint;

6. TRUNCATE : to delete

table's data

TRUNCATE TABLE table-name;